

Importing Data Using Transform Maps

Project Documentation | Full Stack Development Format

Platform: ServiceNow (adapted from the MERN documentation format)

Team Members: see Section 1

1. Introduction

Project Title: Importing Data into ServiceNow Using Transform Maps (Spreadsheet)

Team Members:

Name	Role	Responsibility
N.logeswari	Team Lead / Platform Developer	Data source, transform map configuration
s.christiena	Data & Script Developer	Spreadsheet preparation, transform scripts
N.logeswari	Tester / Documentation	Test cases, screenshots, report

Note: The MERN format is written for web applications. This project is built on ServiceNow, so each MERN section has been mapped to its ServiceNow equivalent: React frontend becomes the platform UI, Node/Express backend becomes the import and transform engine, and MongoDB becomes the platform tables.

2. Project Overview

Purpose

The goal is to import spreadsheet data into a ServiceNow table in a controlled, repeatable way. Instead of typing records by hand, an administrator attaches an Excel file to a data source, loads it into a staging table and runs a transform map that writes the records into the target table, avoiding duplicates and bad rows.

Features

- Excel (.xlsx) or CSV file import through a Data Source.
- Automatic creation of an import set (staging) table.
- Transform map with field mapping, Auto Map and Mapping Assist.
- Coalesce on a unique field (email) to update instead of duplicate.
- onBefore transform script to skip incomplete rows.
- Per-row status: inserted, updated, ignored, error.
- Transform History and Import Log for audit and troubleshooting.
- Optional REST access through the Import Set API.

3. Architecture

Solution Architecture - Transform Map Import

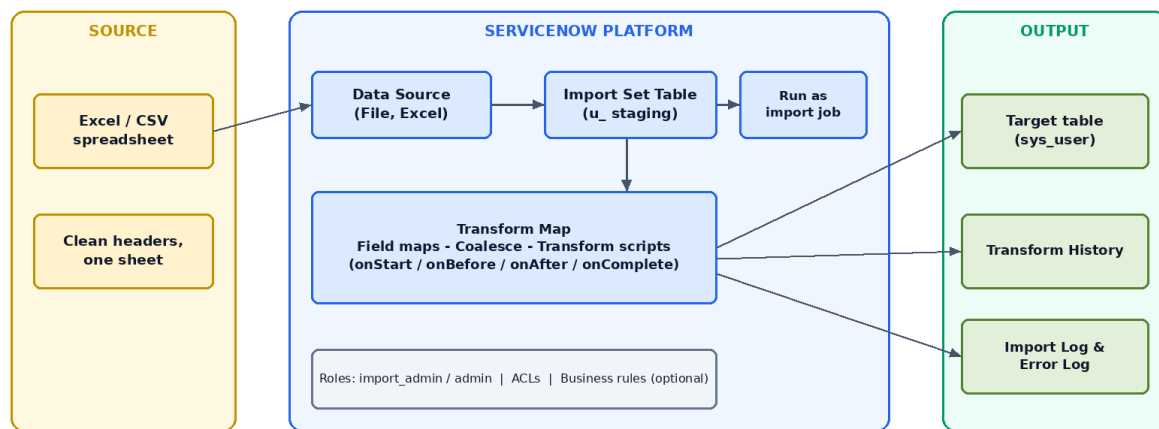


Figure 1: Solution architecture

Frontend

The frontend is the ServiceNow platform UI (Next Experience / Now UI). Administrators work through standard forms and modules instead of a custom React app:

- **System Import Sets > Administration > Data Sources** to define the spreadsheet source.
- **System Import Sets > Load Data** and the **Load All Records** link to stage the rows.
- **System Import Sets > Administration > Transform Maps** to design the mapping and field maps.
- **System Import Sets > Transform History** to review the result of each run.

Backend

The backend is the platform's import and transform engine, which runs server-side. It reads the attached file, writes rows into the import set table, then applies the transform map row by row. Custom logic is written as server-side JavaScript (Glide) in transform scripts such as onStart, onBefore, onAfter and onComplete. The same engine can also be called through the REST **Import Set API**.

Database

ServiceNow stores everything in relational tables. The main ones used here are:

Table	Purpose
sys_data_source	Stores the data source definition and attached file
u_employee_import (u_prefix)	Import set (staging) table created by Load All Records
sys_import_set / sys_import_set_row	Records each import run and every staged row with its status

Table	Purpose
sys_transform_map / sys_transform_entry	Transform map and its field maps
sys_user	Target table that receives the final records
Import log and transform error tables	Messages and errors raised during the run

4. Setup Instructions

Prerequisites

- A ServiceNow instance. A free Personal Developer Instance (PDI) from developer.servicenow.com is enough.
- An account with the admin or import_admin role.
- Microsoft Excel or any tool that can save .xlsx / .csv files.
- A modern browser (Chrome, Edge or Firefox).

Installation

1. Request and sign in to a Personal Developer Instance.
2. Create the sample spreadsheet with the header row: u_first_name, u_last_name, u_email, u_department, u_phone, u_title.
3. Save it as an Excel file with all data on a single sheet.
4. Open **System Import Sets > Administration > Data Sources** and create the data source (Type: File, Format: Excel, Sheet number 1, Header row 1).
5. Attach the spreadsheet and click **Load All Records** to create the import set table.
6. Create the transform map from the import set table to sys_user and add the field maps.
7. Add the onBefore script (see Section 7) and run the transform.

There are no environment variables to configure. Credentials for REST calls are supplied by the caller (see Section 8).

5. Folder Structure

ServiceNow configuration lives on the instance, so the repository is organised around documents, data and exported scripts instead of client/server folders.

Client (platform UI configuration)

```
transform-map-import/
|-- docs/          report, documentation, screenshots
|-- data/          sample spreadsheet (employees.xlsx)
|-- update-set/    exported update set (data source, map, field maps)
```

Server (server-side logic)

```
| -- scripts/
|   |-- onBefore_skip_empty_email.js
|   |-- onAfter_log_summary.js    (optional)
|-- rest-tests/      sample Import Set API requests
|-- README.md
```

6. Running the Application

There is no npm start step. The application is run from the platform:

Step	Where	Action
Frontend	Data Source record	Click Load All Records to stage the spreadsheet
Backend	Import set / Transform Map	Click the Transform related link, choose the map and confirm
Monitor	System Import Sets > Transform History	Check status and counts when the run completes

Equivalent of the start commands: loading the staging table is the 'start frontend' step and running the transform is the 'start backend' step.

7. API Documentation

Imports can also be triggered programmatically through the platform's Import Set API. Replace <instance> with your instance name and u_employee_import with your staging table.

Method	Endpoint	Description
POST	/api/now/import/{stagingTableName}	Insert one record into the staging table and run its transform map
POST	/api/now/import/{stagingTableName}/insertMultiple	Insert multiple records in one call
GET	/api/now/import/{stagingTableName}/{sys_id}	Retrieve a staged record and its transform result
GET	/api/now/table/sys_user?sysparm_query=email=...	Verify the record in the target table (Table API)

Example request

```
POST https://<instance>.service-now.com/api/now/import/u_employee_import
Content-Type: application/json
Accept: application/json

{
  "u_first_name": "Asha",
  "u_last_name": "Kumar",
  "u_email": "asha.kumar@example.com",
```

```
"u_department": "IT"
}
```

Example response (201 Created)

```
{
  "import_set": "ISET0010001",
  "staging_table": "u_employee_import",
  "result": [{
    "transform_map": "Employee Transform",
    "table": "sys_user",
    "display_name": "user_name",
    "display_value": "asha.kumar",
    "record_link": "https://<instance>.service-now.com/api/now/table/sys_user/<sys_id>",
    "status": "inserted"
  }]
}
```

The status value is one of inserted, updated, ignored or error. Exact response fields can vary by release, so confirm them in your instance.

Transform script used by the map

```
(function runTransformScript(source, map, log, target) {
  if (source.u_email.nil()) {
    ignore = true;
    log.info('Row skipped: no email address');
  }
})(source, map, log, target);
```

8. Authentication

- **User login:** Administrators sign in with their ServiceNow user account (optionally via SSO). Sessions are managed by the platform.
- **Authorization:** Access is controlled by roles. The admin or import_admin role is required to create data sources and transform maps and to run transforms. Table access is enforced by ACLs.
- **API access:** REST calls use HTTP Basic authentication over HTTPS or OAuth 2.0 bearer tokens. A dedicated integration user with minimum roles is recommended.
- **Best practice:** Never store passwords in scripts or in the repository. Use credential records or OAuth profiles on the instance.

9. User Interface

The user interface is the standard ServiceNow platform UI. The main screens used in the import are shown below as examples of how they appear in a Personal Developer Instance.

servicenow

Search

dev000000.service-now.com | System Administrator

Data Source - Employee Import Data Source

System Import Sets > Administration > Data Sources

Name

Employee Import Data Source

Import set table label

Employee Import

Import set table name

u_employee_import

Type

File

Format

Excel

File retrieval method

Attachment

Sheet number

1

Header row

1

Attachment

employees.xlsx (12 KB)

Update

Load All Records

Illustrative example - Team ID: SWTID-2026-5741

Data Source form (Type: File, Format: Excel)

servicenow

Search

dev000000.service-now.com | System Administrator

Employee Import [u_employee_import]

System Import Sets > Import Sets > Employee Import

Number	First name	Last name	Email	Department	Phone
IMP0001001	Asha	Kumar	asha.kumar@example.com	IT	98400 10001
IMP0001002	Ravi	Menon	ravi.menon@example.com	HR	98400 10002
IMP0001003	Meena	Iyer	meena.iyer@example.com	Finance	98400 10003
IMP0001004	Karthik	Raj	karthik.raj@example.com	IT	98400 10004
IMP0001005	Divya	Nair	divya.nair@example.com	Support	98400 10005

1 to 5 of 5 rows

State: Loaded | Import set: ISET0010001 | Related link: Transform (choose Employee Transform)

Illustrative example - Team ID: SWTID-2026-5741

Import set table after Load All Records

Table Transform Map - Employee Transform

System Import Sets > Administration > Transform Maps

Name: Employee Transform

Order: 100

Source table: Employee Import [u_employee_import]

Target table: User [sys_user]

Active: ☒ true

Run business rules: ☒ true

Enforce mandatory fields: No

Copy empty fields: ☐ false

[Update](#) [Auto Map Matching Fields](#) [Mapping Assist](#)

Related Lists: [Field Maps](#) | [Transform Scripts](#) | [Entity Transforms](#)

Illustrative example - Team ID: SWTID-2026-5741

Transform Map form (source and target tables)

Field Maps - Employee Transform

System Import Sets > Administration > Transform Maps > Field Maps

Source field	Target field	Coalesce	Use source script
u_first_name	first_name	false	false
u_last_name	last_name	false	false
u_email	email	true	false
u_department	department	false	false
u_phone	phone	false	false
u_title	title	false	false

Highlighted row: Coalesce = true on the email field, so an existing user with the same email is updated instead of duplicated.

Illustrative example - Team ID: SWTID-2026-5741

Field maps with coalesce on email

10. Testing

Strategy: manual functional testing on a Personal Developer Instance, followed by a re-import test and an error-handling test. API checks can be made with Postman.

Test	Method / tool	Expected result
Load spreadsheet	Load All Records	All rows appear in the staging table
Insert new users	Run transform	Rows inserted into sys_user
Duplicate prevention	Run transform again	Rows updated, not duplicated (coalesce on email)
Incomplete row	onBefore script	Row ignored and logged
REST import	Postman, Import Set API	201 response with status inserted

Test	Method / tool	Expected result
Error handling	Invalid data	Error shown in Transform History / error log

Record actual results for your own test run: [Pass / Fail].

11. Screenshots or Demo

servicenow Search dev000000.service-now.com | System Administrator

Transform History - Employee Transform System Import Sets > Transform History

Number	Transform map	State	Inserted	Updated	Ignored	Errors
TMH0001001	Employee Transform	Complete	5	0	0	0

Second run after adding one row with a blank email:

Number	Transform map	State	Inserted	Updated	Ignored	Errors
TMH0001002	Employee Transform	Complete	0	5	1	0

Re-running the same data updates the records (coalesce on email) and the incomplete row is ignored by the onBefore script.

Illustrative example - Team ID: SWTID-2026-5741

Transform History showing a completed run

servicenow Search dev000000.service-now.com | System Administrator

Users [sys_user] User Administration > Users

User ID	Name	Email	Department	Title
asha.kumar	Asha Kumar	asha.kumar@example.com	IT	Analyst
ravi.menon	Ravi Menon	ravi.menon@example.com	HR	Coordinator
meena.iyer	Meena Iyer	meena.iyer@example.com	Finance	Accountant
karthik.raj	Karthik Raj	karthik.raj@example.com	IT	Developer
divya.nair	Divya Nair	divya.nair@example.com	Support	Agent

1 to 5 of 5 rows (filtered: Created today)

Illustrative example - Team ID: SWTID-2026-5741

Imported users in the sys_user table

Demo video: [Paste link here] (file: Import_Data_Using_Transform_Map.mp4)

12. Known Issues

- Merged cells, blank rows or a header that is not on row 1 cause misaligned columns.
- Reference fields (for example department) only match when the value exists in the referenced table.
- Choosing a non-unique coalesce field can overwrite the wrong records.
- Date and number formats in the spreadsheet must match what the target field expects.
- Very large files should be imported in batches or on a schedule to avoid slowing the instance.
- Personal Developer Instances hibernate when idle and must be woken before use.

13. Future Enhancements

- Scheduled imports from an SFTP location using a MID Server.
- Self-service upload widget in the Service Portal.
- Automatic email notification with the import summary.
- Flow Designer integration to trigger imports from approvals.
- Pre-import validation to reject bad rows before staging.
- Extend the template to assets, groups and locations.